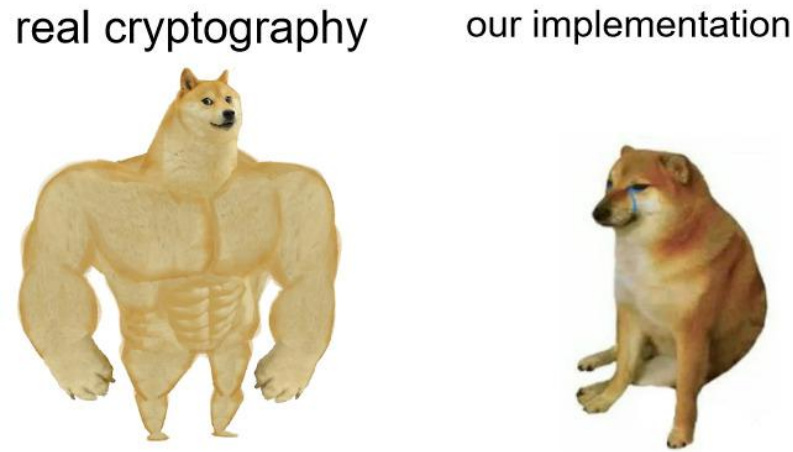
Assignment 1

Pseudo-Random Generator and Stream Cipher

https://ece432-spring26.github.io/assignments/random/index.html

Disclaimer - Never Roll Your Own Cryptography

I’m not a cryptographer,
and neither are you.



https://security.stackexchange.com/a/18198

Assignment One: Roll Your Own Cryptography

To most effectively use cryptographic implementations, you should know the fundamental properties of their implementation.

When you finish A1-3

- Specifications are on the website.
- Submit your solution to gradescope.
- Grade based on autograder and manual inspection.
- You will only see basic sanity tests until after due date.
  - Autograder shows you the easier tests
- For every requirement in the specification, try to write at least one test (where reasonable).

Grading/Submission Requirements

60% PRGen.java

- Deterministically seeded
- Pseudo-random output
- Forward secure (25%)

30% StreamCipher.java

- Implemented using PRGen
- Encrypt satisfying the secure encryption game
- Incorporate nonce to provide correct security properties described in class

10% Style

- Short yet meaningful variable names: avoid one-letter vars (except for loops/counters)
- Use comments judiciously: explain complex or hard-to-understand parts
- Proper indentation: see the Java standard

PRGen Construction from PRF

PRF Interface (implemented for you):

```
public PRF(byte [] prfKey);
public void update(byte[] inBuf, int inOffset, int numBytes)
public void update(byte[] inBuf);
public void eval(byte[]  inBuf, int  inOffset, int numBytes, byte[] outBuf, int outOffset);
public byte[] eval(byte[] inBuf, int inOffset, int numBytes);
public byte[] eval(byte[] val);
```

```
f(k, m) = n
  => PRF f = new PRF(k);
  => byte[] n = f.eval(m);
```

PRGen Interface (for you to implement):

```
public PRGen(byte [] key);
protected int next(int bits);
```

Note: PRGen gives you access to the Java Random interface, which you'll need to use. https://docs.oracle.com/javase/8/docs/api/java/util/Random.html

```
PRGen prg = new PRGen(key);
int i = prg.nextInt();
```

PRGen Implementation Pitfalls

- Forward secrecy
  - Can the state of the PRG reveal previous values?
  - “State” includes any members or instance variables (private or public) of the PRGen class
- Revealing state
  - Does the output reveal any information about state?

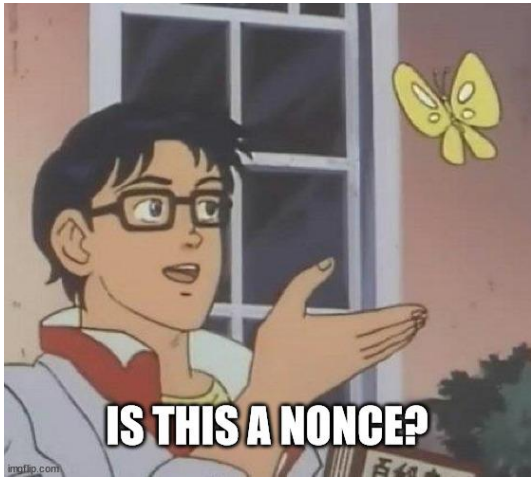
StreamCipher Implementation from PRGen

StreamCipher Interface (for you to implement):

```
public StreamCipher(byte[] key, byte[] nonceArr, int nonceOffset);
public StreamCipher(byte[] key, byte[] nonce);
public byte cryptByte(byte in);
public void cryptBytes(byte[] inBuf, int inOffset, byte[] outBuf, int outOffset, int numBytes);
```

StreamCipher Usage:

```
// encrypt
StreamCipher sc = new StreamCipher(encryptKey, encryptNonce);
byte[] plaintext = "All your base are belong to us!".toByteArray();
byte[] ciphertext = new byte[plaintext.length];
sc.cryptBytes(plaintext, 0, ciphertext, 0, ciphertext.length);
// decrypt (decryptKey = encryptKey, decryptNonce = encryptNonce)
StreamCipher sc = new StreamCipher(decryptKey, decryptNonce);
byte[] recovered = new byte[plaintext.length];
sc.cryptBytes(ciphertext, 0, recovered, 0, recovered.length);
assertArrayEquals(plaintext, recovered);
```



StreamCipher Requirements

- Ciphertext must be random (indistinguishable from random streams).
- Two encryptions of the same plaintext should yield different ciphertexts.
- Any message should be decryptable, even out of order, provided the key.

Ideas:

- Let the PRGen provide randomness; how do we decrypt things out of order?
- Seed the PRGen with the key; how do we differ ciphertext of same plaintext?
- Use a nonce?
  - Nonce: a non-secret, random “number used once”
  - Can be shared to re-sync state in case of message loss: just create a new StreamCipher
  - If each message accompanied by nonce, can be encrypted/decrypted out of order
  - Prevents replay attacks

StreamCipher - Session Keys

Cryptographically **combining** a key with a message-specific nonce is one way to create a “session key.” Each message has a unique and random nonce, so each message is encrypted/decrypted from the beginning of a PRGen stream.

```
// create nonce for session key (assume encryptKey == decryptKey has been pre-shared)
byte[] messageNonce = new byte[StreamCipher.NONCE_SIZE_BYTES];
new SecureRandom().nextBytes(messageNonce);
// encrypt
StreamCipher sc = new StreamCipher(encryptKey, messageNonce);
byte[] plaintext = "All your base are belong to us!".toByteArray();
byte[] ciphertext = new byte[plaintext.length];
sc.cryptBytes(plaintext, 0, ciphertext, 0, ciphertext.length);
// decrypt (imagine sending the cipherText along with the messageNonce)
StreamCipher sc = new StreamCipher(decryptKey, messageNonce);
byte[] recovered = new byte[plaintext.length];
sc.cryptBytes(ciphertext, 0, recovered, 0, recovered.length);
assertArrayEquals(plaintext, recovered);
```